

```

import os
import re
import random
import asyncio
import threading
import traceback

import wikipediaapi
from flask import Flask

from telethon import TelegramClient, events
from telethon.sessions import StringSession
from telethon.tl.types import MessageEntityMentionName, MessageEntityMention

from supabase import create_client
from gigachat import GigaChat
from gigachat.models import Chat, Messages, MessagesRole

# =====
# КОНФИГ
# =====
API_ID = int(os.environ['TELEGRAM_API_ID'])          # my.telegram.org
API_HASH = os.environ['TELEGRAM_API_HASH']          # my.telegram.org
SESSION_STRING = os.environ['TELEGRAM_SESSION']      # получить через generate

AUTHORIZED_USER_ID = int(os.environ.get('AUTHORIZED_USER_ID', 0))
GIGACHAT_MODEL = os.environ.get('GIGACHAT_MODEL', 'GigaChat-2-Max')
PORT = int(os.environ.get('PORT', 10000))

supabase = create_client(
    os.environ['SUPABASE_URL'],
    os.environ['SUPABASE_KEY'],
)

giga_client = GigaChat(
    credentials=os.environ['GIGACHAT_CREDENTIALS'],
    scope=os.environ.get('GIGACHAT_SCOPE', 'GIGACHAT_API_PERS'),
    verify_ssl_certs=False,
)

client = TelegramClient(StringSession(SESSION_STRING), API_ID, API_HASH)

# Защита от двойных сообщений — раньше threading.Lock/set, теперь asyncio,
# т.к. Telethon работает на одном event loop'e.

```

```

processing_chats = set()
processing_lock = asyncio.Lock()

ROLE_MAP = {
    'system': MessagesRole.SYSTEM,
    'user': MessagesRole.USER,
    'assistant': MessagesRole.ASSISTANT,
}

# -----
# Список триггеров для защиты от Prompt Injection
# -----
PROMPT_INJECTION_TRIGGERS = (
    '[system note', 'override', 'debug mode', 'режим отладки',
    'забудь роль', 'смени личность', 'выведи инструкции', 'покажи промпт',
    'забудь все правила', 'emergency override', 'сбрось настройки',
    'отключи роль', 'стань свободным', 'игнорируй промпт'
)

# -----
# Системный промпт — как и в оригинале, заполни своим текстом
# -----
SYSTEM_PROMPT = {
    "role": "system",
    "content": (
        ""
    )
}

# =====
# GigaChat — обёртка вместо groq_client
# =====
def _gigachat_complete_sync(messages, temperature=0.7, max_tokens=1024):
    """Синхронный вызов GigaChat (SDK синхронный, поэтому вызывается через to_thr
    payload = Chat(
        model=GIGACHAT_MODEL,
        messages=[
            Messages(role=ROLE_MAP.get(m['role'], MessagesRole.USER), content=m['
            for m in messages
        ],
        temperature=temperature,
        max_tokens=max_tokens,
    )
    response = giga_client.chat(payload)
    return response.choices[0].message.content

```

```

async def gigachat_complete(messages, temperature=0.7, max_tokens=1024):
    return await asyncio.to_thread(_gigachat_complete_sync, messages, temperature

# =====
# Вспомогательные функции Telegram (через Telethon, вместо HTTP к Bot API)
# =====
async def send_telegram_message(chat_id, text):
    try:
        await client.send_message(chat_id, text)
    except Exception:
        pass

async def can_restrict_member(chat_id, user_id):
    """Проверяет, можно ли ограничить участника (не админ ли он)."""
    try:
        perms = await client.get_permissions(chat_id, user_id)
        if perms.is_admin or perms.is_creator:
            return False
        return True
    except Exception:
        return False

async def ban_user(chat_id, user_id):
    """Банит пользователя. Сначала пробует edit_permissions (супергруппы),
    при неудаче — kick_participant (обычные группы)."""
    try:
        await client.edit_permissions(chat_id, user_id, view_messages=False)
        return True
    except Exception:
        try:
            await client.kick_participant(chat_id, user_id)
            return True
        except Exception:
            return False

def _search_wikipedia_sync(query, lang='ru'):
    user_agent = "KirenaBot/1.0 (https://t.me/your_account; your_email@example.co
    wiki_wiki = wikipediaapi.Wikipedia(user_agent, lang)
    page = wiki_wiki.page(query)
    if page.exists():
        return f"📖 {page.title}\n{page.summary[0:200]}\n🔗 {page.fullurl}"
    return f"😞 К сожалению, я не нашла статью по запросу «{query}». Попробуй пер

```

```

async def search_wikipedia(query, lang='ru'):
    return await asyncio.to_thread(_search_wikipedia_sync, query, lang)

# =====
# Работа с историей диалога (Supabase) – синхронный клиент,
# все вызовы завернуты в asyncio.to_thread, чтобы не блокировать event loop
# =====
def _load_history_sync(chat_id):
    data = supabase.table('users').select('history').eq('chat_id', chat_id).execu
    history = data.data[0].get('history', []) if data.data else []

    chat_type = 'private' if chat_id > 0 else 'group'
    system_content = SYSTEM_PROMPT['content']

    other_facts = _load_global_facts_sample_sync(chat_id, chat_type, limit=5)
    if other_facts:
        facts_block = (
            "Факты о людях, с которыми я общался "
            "(используй, если уместно, но **никогда не раскрывай личную "
            "информацию из приватных бесед в группе**):\n"
        )
        facts_block += "\n".join(f"- {fact}" for fact in other_facts)
        system_content += "\n\n" + facts_block

    if chat_type == 'group':
        system_content += (
            "\n\nТы находишься в групповом чате. "
            "Любые факты, помеченные как личные, не должны упоминаться здесь, "
            "даже если они относятся к кому-то из участников."
        )

    user_info = f"\nТы сейчас общаешься с пользователем chat_id = {chat_id}."
    if chat_type == 'group':
        user_info += " Это групповой чат. Обращайся к людям по именам, если знаешь"
    system_content += user_info

    system_msg = {"role": "system", "content": system_content}

    if history and history[0].get('role') == 'system':
        history[0] = system_msg
    else:
        history.insert(0, system_msg)

    return history

```

```

async def load_history(chat_id):
    return await asyncio.to_thread(_load_history_sync, chat_id)

def _save_history_sync(chat_id, history):
    history_to_save = [msg for msg in history if msg.get('role') != 'system']
    supabase.table('users').upsert({'chat_id': chat_id, 'history': history_to_save})

async def save_history(chat_id, history):
    await asyncio.to_thread(_save_history_sync, chat_id, history)

# -----
# Сжатие истории
# -----
def _build_summary_prompt(history_chunk):
    transcript = ""
    for msg in history_chunk:
        if msg['role'] == 'system':
            continue
        role = "Пользователь" if msg['role'] == 'user' else "Бот"
        transcript += f"{role}: {msg['content']}\n"
    return (
        "Сделай очень краткое резюме этого диалога (2-3 предложения), "
        "сохранив ключевые факты и договорённости:\n" + transcript
    )

async def summarize_text(history_chunk):
    prompt = _build_summary_prompt(history_chunk)
    return await gigachat_complete([{"role": "user", "content": prompt}], tempera

async def compress_history(history, keep_last=5, max_messages=18):
    if len(history) <= max_messages:
        return history

    system_msgs = [msg for msg in history if msg['role'] == 'system']
    dialog_msgs = [msg for msg in history if msg['role'] != 'system']

    if len(dialog_msgs) <= keep_last:
        return history

    old_part = dialog_msgs[:-keep_last]

```

```

recent_part = dialog_msgs[-keep_last:]

summary = await summarize_text(old_part)
summary_msg = {"role": "system", "content": f"[Резюме предыдущего разговора]:"}

return system_msgs + [summary_msg] + recent_part

# -----
# Оценка серьёзности причины кика
# -----
async def evaluate_kick_reason(reason_text):
    prompt = (
        "Ты – Кирена, добрая и миролюбивая помощница. Тебя попросили исключить че  

        "Ты должна оценить, насколько указанная причина действительно заслуживает  

        "Серьёзными считаются: спам, оскорбления, угрозы, распространение порногр  

        "преследование участников, явное нарушение правил чата.\n"
        "Несерьёзными считаются: личная неприязнь, «он мне не нравится», «просто  

        f"Причина: \"{reason_text}\".\n\n"
        "Ответь только одно слово: \"серьёзно\" или \"несерьёзно\"."
    )
    try:
        result = await gigachat_complete([{"role": "user", "content": prompt}], t
        return 'серьёзно' in result.strip().lower()
    except Exception:
        return False

# -----
# Общая память (автономное извлечение фактов)
# -----
async def extract_facts_with_context(history_before_answer, user_message, chat_id
    recent_history = history_before_answer[-10:] if len(history_before_answer) >

    transcript = ""
    for msg in recent_history:
        role = "Пользователь" if msg['role'] == 'user' else "Бот"
        transcript += f"{role}: {msg['content']}\n"

    prompt = (
        "Проанализируй диалог и выдели факты о пользователе.\n"
        "ВАЖНО: Если пользователь явно сказал, что какую-то информацию МОЖНО или  

        "рассказывать другим, обязательно учти это при оценке приватности.\n\n"
        "Формат для каждого факта (на новой строке):\n"
        "факт | true/false | обоснование\n\n"
        "где true – личное (не рассказывать), false – можно рассказывать.\n"
        "Обоснование – краткая причина твоего решения.\n\n"

```

```

        f"Диалог:\n{transcript}\n"
        f"Последнее сообщение пользователя: \"{user_message}\"\\n\\n"
        "Факты с оценкой:"
    )

    content = await gigachat_complete([{"role": "user", "content": prompt}], temp
    content = (content or "").strip()
    if not content:
        return []

    facts = []
    for line in content.split('\n'):
        line = line.strip()
        if '|' in line:
            parts = line.split('|')
            if len(parts) >= 2:
                fact_text = parts[0].strip()
                is_private = parts[1].strip().lower() == 'true'
                if fact_text:
                    facts.append({'fact': fact_text, 'is_private': is_private})
    return facts

def _save_global_facts_sync(facts, chat_id, chat_type):
    for f in facts:
        supabase.table('global_facts').insert({
            'fact_text': f['fact'],
            'source_chat_id': chat_id,
            'is_private': f['is_private'],
            'chat_type': chat_type
        }).execute()

async def save_global_facts(facts, chat_id, chat_type):
    await asyncio.to_thread(_save_global_facts_sync, facts, chat_id, chat_type)

def _load_global_facts_sample_sync(current_chat_id, chat_type, limit=5):
    if chat_type == 'private':
        resp = (
            supabase.table('global_facts')
            .select('fact_text', 'is_private', 'source_chat_id', 'chat_type')
            .or_(f'is_private.eq.false, and(is_private.eq.true,source_chat_id.eq.
            .order('created_at', desc=True)
            .limit(30)
            .execute()
        )

```

```

else:
    resp = (
        supabase.table('global_facts')
        .select('fact_text', 'is_private', 'source_chat_id', 'chat_type')
        .eq('is_private', False)
        .order('created_at', desc=True)
        .limit(30)
        .execute()
    )
    facts = resp.data
    if not facts:
        return []
    sample = random.sample(facts, min(limit, len(facts)))
    return [f['fact_text'] for f in sample]

# =====
# Supabase-хелперы для kick_requests (обёрнуты в to_thread)
# =====
async def db_delete_kick_request(chat_id, requester_id):
    await asyncio.to_thread(
        lambda: supabase.table('kick_requests').delete()
        .eq('chat_id', chat_id).eq('requester_id', requester_id).execute()
    )

async def db_get_pending_kick(chat_id, requester_id):
    resp = await asyncio.to_thread(
        lambda: supabase.table('kick_requests').select('*')
        .eq('chat_id', chat_id).eq('requester_id', requester_id).execute()
    )
    return resp.data

async def db_get_pending_kick_no_target(chat_id, requester_id):
    resp = await asyncio.to_thread(
        lambda: supabase.table('kick_requests').select('*')
        .eq('chat_id', chat_id).eq('requester_id', requester_id)
        .is_('target_id', 'null').execute()
    )
    return resp.data

async def db_insert_kick_request(chat_id, requester_id, target_id):
    await asyncio.to_thread(
        lambda: supabase.table('kick_requests').insert({
            'chat_id': chat_id, 'requester_id': requester_id, 'target_id': target

```

```

        }).execute()
    )

async def db_update_kick_target(request_id, target_id):
    await asyncio.to_thread(
        lambda: supabase.table('kick_requests').update({'target_id': target_id}).
    )

async def db_clear_user(chat_id):
    await asyncio.to_thread(lambda: supabase.table('users').delete().eq('chat_id'
    await asyncio.to_thread(lambda: supabase.table('global_facts').delete().eq('s
    try:
        await asyncio.to_thread(lambda: supabase.table('style_examples').delete()
    except Exception:
        pass

# =====
# Извлечение упомянутого пользователя из сообщения (Telethon)
# =====
async def extract_mentioned_user(message):
    """Возвращает (user_id, отображаемый_текст) или (None, 'участника')."""
    try:
        entities = message.get_entities_text() or []
    except Exception:
        entities = []

    for ent, txt in entities:
        if isinstance(ent, MessageEntityMentionName):
            return ent.user_id, txt
        if isinstance(ent, MessageEntityMention):
            try:
                user = await client.get_entity(txt)
                return user.id, txt
            except Exception:
                return None, txt
    return None, "участника"

# =====
# Основной обработчик входящих сообщений
# =====
@client.on(events.NewMessage(incoming=True))
async def handle_message(event):
    if event.out:

```

```

        return

chat_id = event.chat_id
user_id = event.sender_id
text = event.raw_text or ''

# --- Защита от Prompt Injection ---
if text:
    text_lower = text.lower()
    if any(trigger in text_lower for trigger in PROMPT_INJECTION_TRIGGERS):
        await send_telegram_message(chat_id, "Извини, я не могу это сделать.")
        return

# --- Отмена запроса на исключение ---
if text and any(phrase in text.lower() for phrase in ['кира, отмени исключени
    await db_delete_kick_request(chat_id, user_id)
    await send_telegram_message(chat_id, "Запрос на исключение отменён.")
    return

# --- Ожидание причины исключения ---
if chat_id < 0:
    pending = await db_get_pending_kick(chat_id, user_id)
    if pending:
        reason = text.strip()
        target_id = pending[0]['target_id']
        await db_delete_kick_request(chat_id, user_id)

        if await evaluate_kick_reason(reason):
            if await can_restrict_member(chat_id, target_id):
                success = await ban_user(chat_id, target_id)
                if success:
                    await send_telegram_message(chat_id, f"Готово. Пользовате
                else:
                    await send_telegram_message(chat_id, "Не удалось исключить
            else:
                await send_telegram_message(chat_id, "Я не могу исключить это
        else:
            await send_telegram_message(
                chat_id,
                f"Извини, но причина «{reason}» недостаточно серьёзна, чтобы
                "Нужно что-то вроде спама, оскорблений или угроз."
            )
    return

# --- Обнаружение намерения исключить ---
kick_triggers = ['исключи', 'забань', 'выгони', 'кикни', 'кик', 'заблокируй',
if text and any(trigger in text.lower() for trigger in kick_triggers):

```

```

target_id, target_mention = await extract_mentioned_user(event.message)

if not target_id:
    await db_insert_kick_request(chat_id, user_id, None)
    await send_telegram_message(chat_id, "Кого именно исключить? Пожалуйста")
    return

await db_insert_kick_request(chat_id, user_id, target_id)
await send_telegram_message(chat_id, f"За что исключить {target_mention}?")
return

# --- Ответ с @username, когда ждали уточнения цели ---
if chat_id < 0 and text and not any(trigger in text.lower() for trigger in ki
    pending = await db_get_pending_kick_no_target(chat_id, user_id)
    if pending:
        target_id, target_mention = await extract_mentioned_user(event.message)
        if target_id:
            await db_update_kick_target(pending[0]['id'], target_id)
            await send_telegram_message(chat_id, f"За что исключить {target_m
        else:
            await send_telegram_message(chat_id, "Я всё ещё жду @username. Уп
    return

# --- /poem ---
if text.startswith('/poem'):
    topic = text[6:].strip() or "о чём-нибудь прекрасном"
    await send_telegram_message(chat_id, f"Сейчас сочиню что-нибудь {topic}..
    text = f"Напиши короткое стихотворение {topic}. Без вступления и пояснени

# --- /clear ---
if text == '/clear':
    await db_clear_user(chat_id)
    await send_telegram_message(chat_id, "🗑️ Всё забыто (наверн). Начинаем с
    return

# --- /start ---
if text == '/start':
    await send_telegram_message(chat_id, "Привет! Я Кирена")
    return

# --- Защита от двойных сообщений ---
lock_key = (chat_id, user_id)
async with processing_lock:
    if lock_key in processing_chats:
        try:
            await event.message.delete()
        except Exception:

```

```

        if chat_id > 0:
            await send_telegram_message(chat_id, "Кирена пока занята отве
        return
    processing_chats.add(lock_key)

try:
    # --- Wikipedia ---
    if text and '[WIKI:]' in text:
        start = text.find('[WIKI:]') + 6
        end = text.find(']', start)
        if end != -1:
            query = text[start:end].strip()
            wiki_response = await search_wikipedia(query)
            await send_telegram_message(chat_id, wiki_response)
            return

    # --- Редактирование кода (только для владельца) ---
    if text and '[EDIT:]' in text:
        if user_id != AUTHORIZED_USER_ID:
            await send_telegram_message(chat_id, "🚫 Извини, но редактировать
            return

        start = text.find('[EDIT:]') + 6
        end = text.find(']', start)
        if end != -1:
            params = text[start:end].split('|')
            if len(params) >= 3:
                file_path = params[0].strip()
                await send_telegram_message(chat_id, f"✅ Команда на редактир
            else:
                await send_telegram_message(chat_id, "Формат: `[EDIT: путь_к_
            else:
                await send_telegram_message(chat_id, "Неверный формат команды.")
            return

    # --- Основной диалог через GigaChat ---
    history = await load_history(chat_id)
    history.append({"role": "user", "content": text})
    history_before_answer = history.copy()

    async with client.action(chat_id, 'typing'):
        raw_answer = await gigachat_complete(history, temperature=0.7, max_to

    raw_answer = re.sub(r'<\s*think\s*>.*?<\s*/\s*think\s*>', '', raw_answer,
    raw_answer = re.sub(r'<\s*think\s*>.*$', '', raw_answer, flags=re.DOTALL
    raw_answer = re.sub(r'<\s*/\s*think\s*>', '', raw_answer, flags=re.IGNORE
    raw_answer = re.sub(r'<[^>]+>', '', raw_answer)

```

```

        answer = '\n'.join(line.strip() for line in raw_answer.split('\n') if line)
        if not answer:
            answer = "Не удалось получить ответ. Попробуй ещё раз."

        history.append({"role": "assistant", "content": answer})
        history = await compress_history(history, keep_last=5, max_messages=22)
        await save_history(chat_id, history)

        if text and text != '/start':
            chat_type = 'private' if chat_id > 0 else 'group'
            facts = await extract_facts_with_context(history_before_answer, text,
            if facts:
                await save_global_facts(facts, chat_id, chat_type)

        await send_telegram_message(chat_id, answer)

    except Exception as e:
        error_str = str(e)
        await send_telegram_message(chat_id, f"Ошибка: {error_str}")
        if AUTHORIZED_USER_ID:
            detail = (
                f"⚠ Ошибка у Киры:\n{error_str}\nЧат: {chat_id}\nТекст: {text[:200]}\n"
                f"Трассировка:\n{traceback.format_exc()}"
            )
            if len(detail) > 4000:
                detail = detail[:4000]
            await send_telegram_message(AUTHORIZED_USER_ID, detail)
    finally:
        async with processing_lock:
            processing_chats.discard(lock_key)

# =====
# Мини health-check сервер — нужен только чтобы Render (Web Service)
# не считал контейнер мёртвым (порт должен отвечать на HTTP).
# Если разворачиваешь как Background Worker — этот блок не нужен.
# =====
health_app = Flask(__name__)

@health_app.route('/')
def health():
    return 'OK'

def run_health_server():
    health_app.run(host='0.0.0.0', port=PORT)

```

```
def main():
    threading.Thread(target=run_health_server, daemon=True).start()
    with client:
        print("Userbot запущен.")
        client.run_until_disconnected()

if __name__ == '__main__':
    main()
```